轮 趣 科 技

C63A 主控用户手册

推荐关注我们的公众号获取更新资料

版本说明：

版本	日期	内容说明
V1.0	2023/12/18	第一次发布

网址:www.wheeltec.net

序言

本教程的内容主要针 C63A 主板的使用教程，C63A 主板使用了 STM32F407VET6 主控芯片，外设丰富，性能强大。

手册中涵盖使用过程中出现的大部分问题，若遇到问题，请先仔细阅读手册。

目录

序言 2

1. 参数说明 4

2. 快速验证 5

3. 主板外设接口接线说明 6

3.1 航模遥控 7

3.2 超声波 7

3.3 RGB 灯带接口 7

3.4 对外供电 5V5A 接口 8

3.5 串口通信 9

4. 主板交互式接口使用说明 10

4.1 显示屏 10

4.2 蓝牙模块 10

4.3 蜂鸣器 11

4.4 急停开关与使能开关 11

5. 常用代码接口 12

5.1 灯带控制 12

5.2 超声波数据获取 13

5.3 显示屏显示内容 15

6. ROS 通信协议 16

6.1 ROS 接口控制输入协议说明 16

6.2 ROS 接口默认发送的数据协议 16

7. 如何给 STM32 控制器下载代码 24

7.1 串口一键下载 24

7.2 ST-Link 下载 25

1. 参数说明

表 1 参数说明表

主控芯片	STM32F407VET6
可驱动电机类型	支持 CAN 通信接口的任意类型电机驱动
可驱动 PWM 舵机数量	4 个
可驱动超声波数量	6 个
串口预留	3 个
RGB 可调灯带接口	2 个
CAN 接口数量	2 个
485 通信接口	1 个
GPIO 预留	数十个
ROS 底层代码和 ROS SDK	提供
IMU 芯片	ICM20948 九轴传感器
输入电源电压范围	10V-28V
下载接口	支持串口一键下载，预留 SWM 接口
对外供电接口	独立 5V5A 两个，3.3V 和 5V 若干
板子尺寸	75*75mm
板子重量	61g
安装孔径孔距	孔径：3mm 直径，孔距：69*69mm
板子层数	工业级四层板
保护电路	过热保护、短接保护、过流保护
电机使能开关	配备
人机交互开关	按键、蜂鸣器、OLED 屏幕、LED 灯

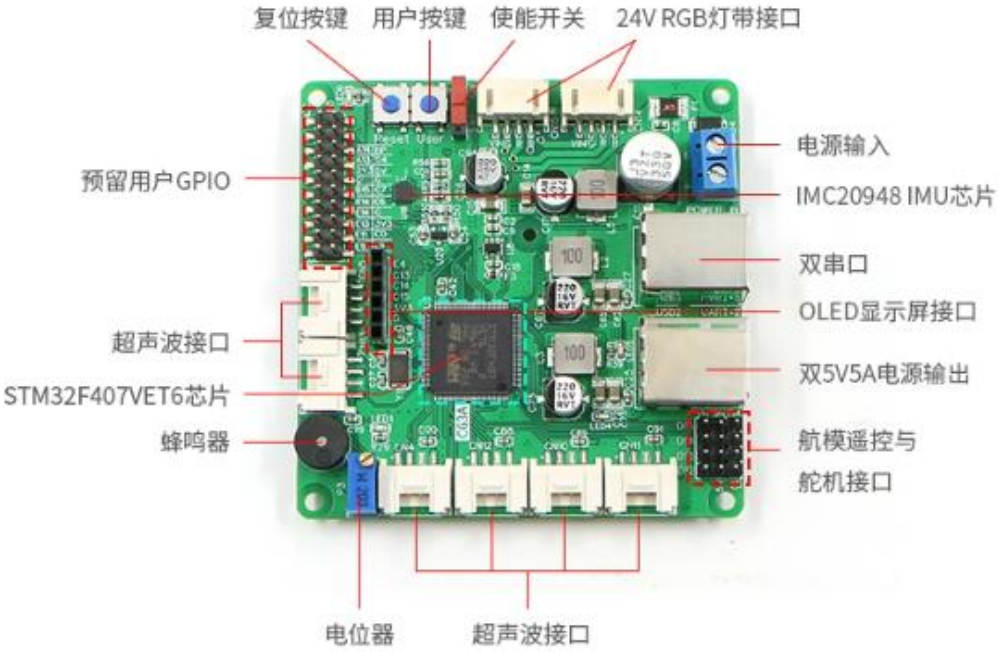
2. 快速验证

图 2-1 主板快速验证数据图示

上电后蜂鸣器会连续响两次，LED6 灯会闪烁，屏幕点亮，可以观察到屏幕中的陀螺仪数据和电压数据。

3. 主板外设接口接线说明

本章内容主要说明 C63A 主板的外设接口接线相关的说明与图示演示以供参考。



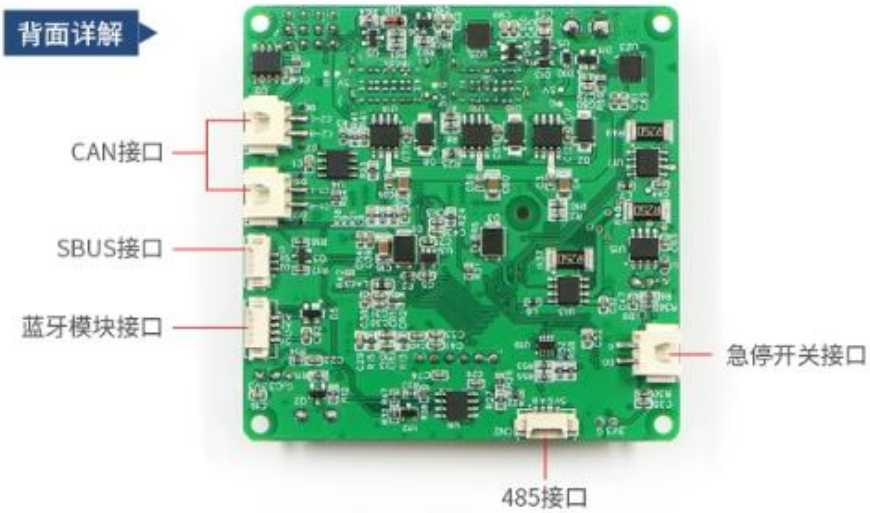


图 3-1 C63A 外设图示

3.1航模遥控

图 3-2 航模接口图示

3.2超声波

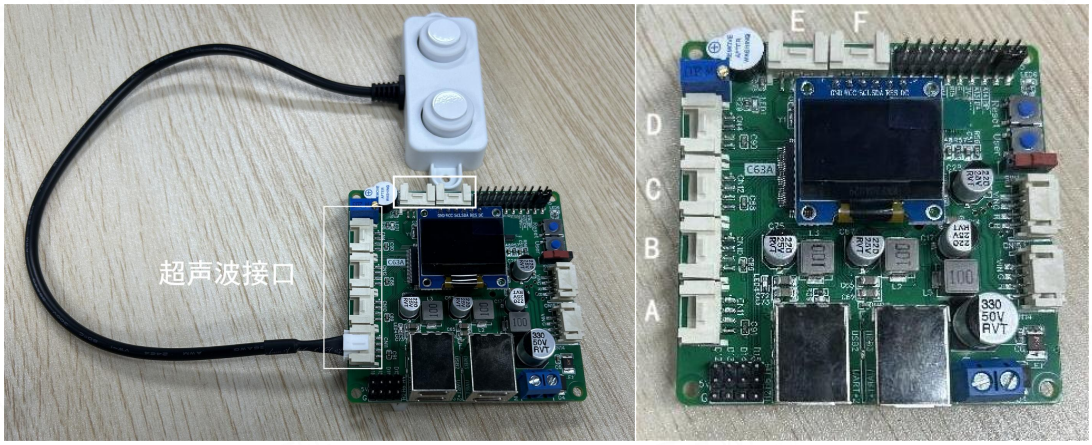


图 3-3 超声波接线顺序图示

3.3RGB 灯带接口

RGB 灯带提供两个接口，两个接口是并联关系，接口线序与我们配套的灯带已经设置好可以直接接入进行使用，若使用自购灯带请注意灯带的线序是否对应上。

RGB 灯的原色为红绿蓝三色，可以通过控制这三种原色的比例让灯带呈现

出任意一种颜色。

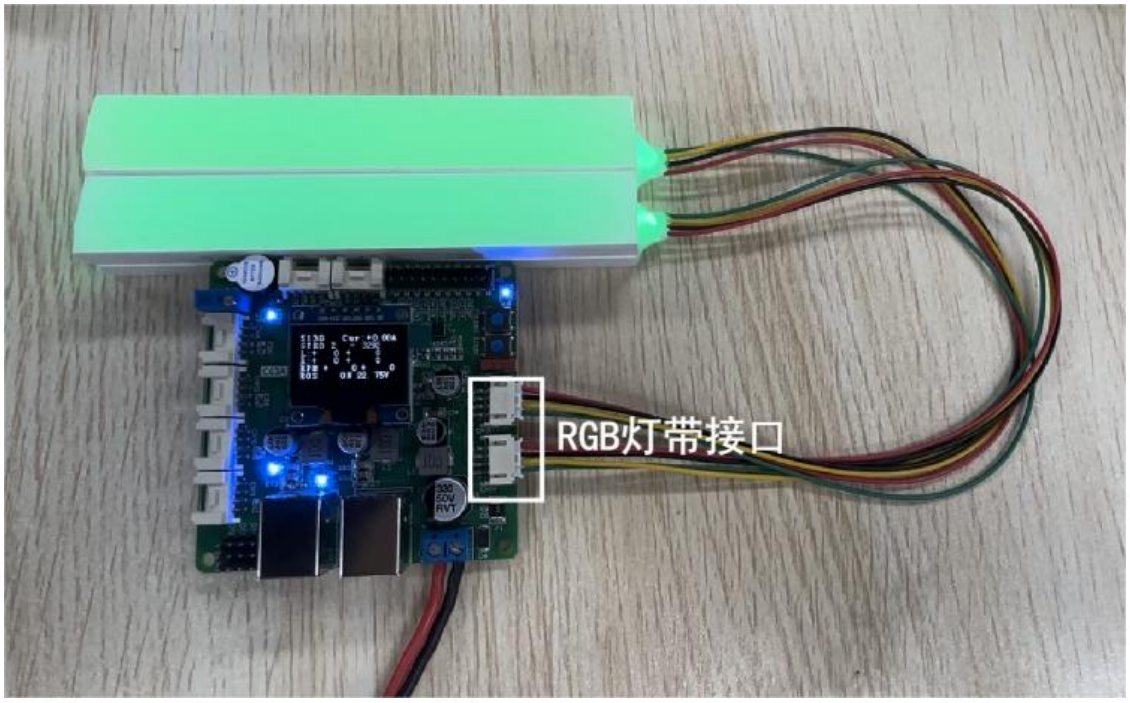


图 3-4 RGB 灯带接口

3.4对外供电 5V5A 接口

两个 TYPE-C 接口分别是独立的 5V5A 电源输出接口，可供电于 JetsonNano、鲁班猫、X3 派、树莓派等开发板。

图 3-5 5V5A 接口图示

3.5串口通信

Type-C 串口通信接口，接口已经内置 TTL 电平转换芯片（CH9102），通信控制使用上接口串口 4，代码下载使用下接口串口 1。

图 3-6 串口通信接口图示

4. 主板交互式接口使用说明

本章内容主要说明 C63A 主板的交互式接口的使用与图示演示以供参考。

4.1 显示屏

通过 OLED 显示屏可以显示出我们需要观察的一些数据，例如我们默认的显示内容，可以观察陀螺仪数据、速度、电压等数据。

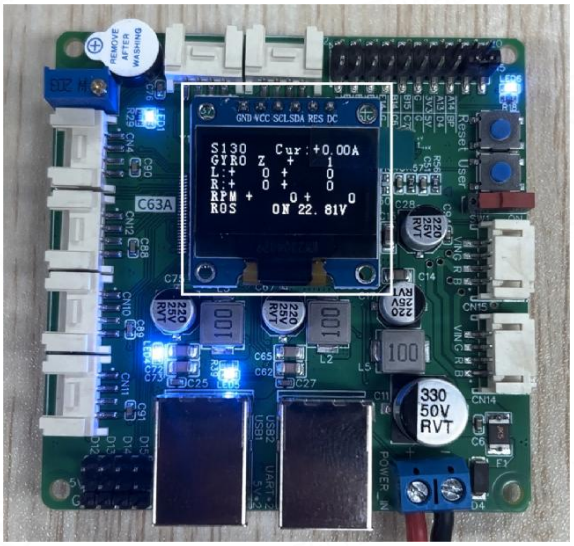


图 4-1 OLED 显示屏图示

4.2 蓝牙模块

蓝牙模块是通过一个 4pin 排线和 C63A 连接，使用的是串口 2，详细的通信内容请参考《APP 通信协议手册》。

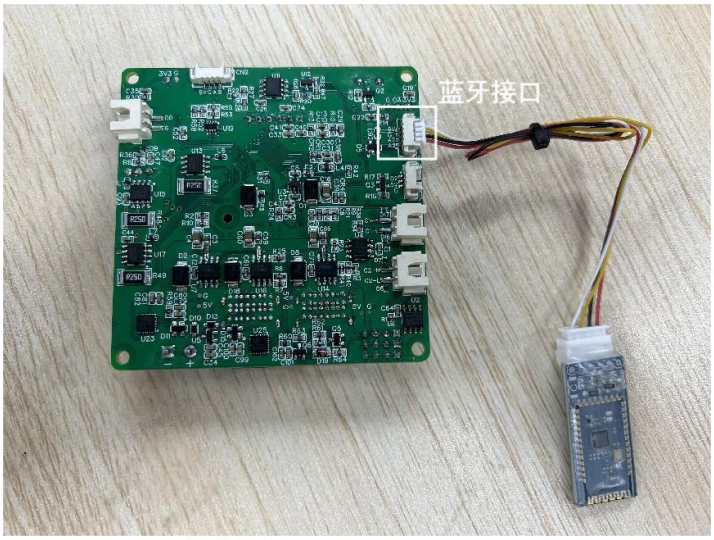


图 4-2 蓝牙模块接口图示

4.3蜂鸣器

C63A 的蜂鸣器需要插上跳线帽才能正常使用，C63A 出厂默认不使能蜂鸣器，如需使用板载蜂鸣器需将跳线帽切换到 BP 和 D4 引脚上。

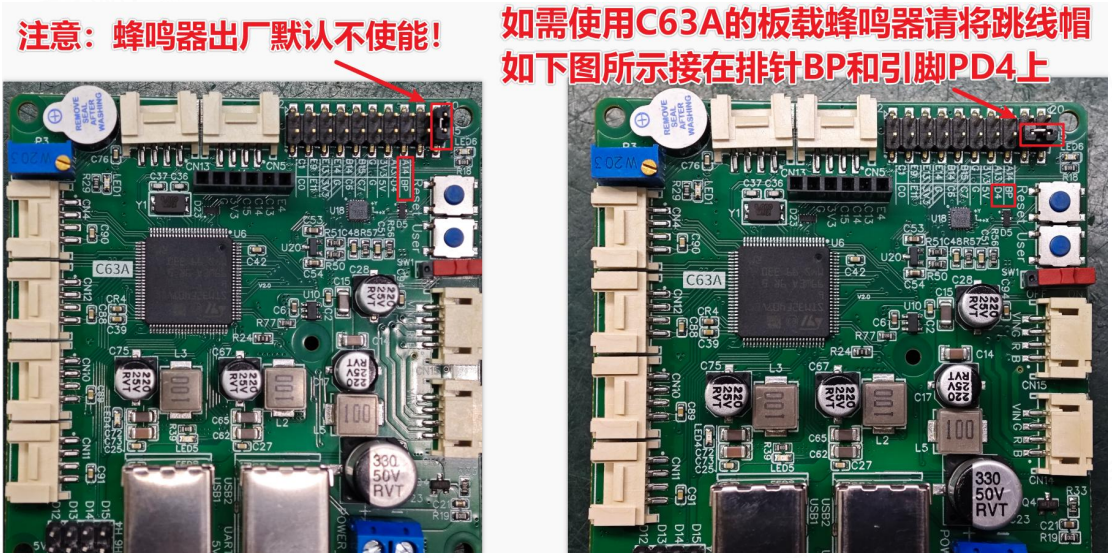


图 4-3 蜂鸣器与其跳线帽图示

4.4急停开关与使能开关

两个外设使用的是同一个引脚，工作原理是该 IO 检测外部电平的变化关断电机驱动的输出，实现软件急停。急停开关接口默认低电平为失能状态，用户使用时需外接开关，开关将两个引脚导通时则时间急停的信号输入。

图 4-4 急停开关与使能开关图示

5. 常用代码接口

本章内容主要说明 C63A 主板常用接口的例程代码，此处代码仅供参考。

5.1灯带控制

RGB 灯带颜色的设置在 LED.c 文件中的 void RGB_Set()函数中，在初始化中将 RGB 初始化为 500Hz 的 PWM 信号，所以 RGB 的设置的范围为 0~255，在 255 时是最亮的一个状态。在我们的代码中，灯带在机器人不同状态下会呈现不同的颜色，详细情况可参考 read_distance.c 文件中的 214 行之后的代码，如图 5-2（部分）所示。我们的代码中主要用到了红绿蓝三种原色，想要呈现其余颜色，需自行进行调试。

```
//灯带颜色设置函数
void RGB_Set(u8 r_value,u8 g_value,u8 b_value)
{
    LED_R = r_value;
    LED_G = g_value;
    LED_B = b_value;
}
```

图 5-1 灯带颜色设置代码图示

下面时使用场景案例：案例 1：在图 5-2 中，设置了机器人在充电大电流时 RGB 灯带为红色，小电流（也就是快充满）时为绿色。修改 rgb_set[i]中的数值大小就可以修改颜色变化。rgb_set[1]是控制红色，rgb_set[2]是控制绿色，rgb_set[3]是控制蓝色。案例 2：在超声波探测到物体距离机器人时正常范围时，RGB 灯是一个在红绿蓝三色中变换的一个呼吸灯状态，如图 5-3 所示，在超声波探测到近距离物体时，RGB 灯会变为绿色，如图 5-4 所示。

```
224         if(rgb_100Hz_control==1) //频率控制在100Hz
225         {
226             rgb_100Hz_control = 0;
227
228             if(rgb_set[0]!=User_defined && SecurityPLY==0 )//用户自定义灯带优先级最高
229             {
230                 if(Allow_Recharge)//自动回充模式
231                 {
232                     rgb_set[0]=Auto_recharge;
233
234                     if(Charging||ChargDelay)//充电状态颜色
235                     {
236                         if(charge_full==0)
237                         {
238                             /* 红色 */
239                             if(rgb_set[1]<255) rgb_set[1]++;
240                             if(rgb_set[2]!=0) rgb_set[2]--;
241                             if(rgb_set[3]!=0) rgb_set[3]--;
242                             rainbow_rgb_reset=1;
243                         }
244                         else
245                         {
246                             /* 绿色 */
247                             if(rgb_set[1]<120) rgb_set[1]++;
248                             if(rgb_set[1]>120) rgb_set[1]--;
249                             if(rgb_set[2]<255) rgb_set[2]++;
250                             if(rgb_set[3]!=0) rgb_set[3]--;
251                             rainbow_rgb_reset=2;
252                         }
253
```

图 5-2 RGB 灯带代码图示

图 5-3 超声波探测物体距离远近时 RGB 灯图示

5.2超声波数据获取

超声波是采用定时器输入捕获模式来测量距离。超声波的最远距离在 3 米，所以在程序中做了限制，在距离超过 3 米或没有超声波模块时，OLED 屏幕显示为 3 米。

超声波的工作原理为：给触发引脚一个至少 10us 的脉冲信号，此时模块会发出超声波，回波引脚会由低电平变为高电平，若模块接受弹回来的超声波，回波引脚就会变为低电平，我们通过测量高电平的持续时间来计算出距离。时序如下图所示。

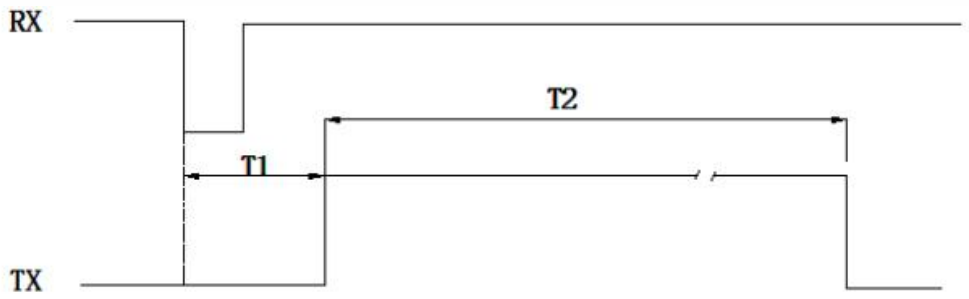


图 5-4 超声波时序图

计算公式：测试距离=（高电平*声速（340/s））/2；

在 read_distance.c 文件中的 void ReadUS_task()函数就是机器人的超声波测距任务，可以看到超声波是分为两组，一组三个进行测距。TriggerX 就是就是触发引脚，可以看到电平是先拉高，等待 12us，再拉低。

```
void US_ABCF_Read(void)
{
    ////////////////  超声波A ////////////////
    if(Get_US_A_State != RESET)//TIM5CH3发生输入捕获事件
    {
        Clear_US_A_State;//清除中断标志位

        if(US_A_FLag==2)//第2次进入中断
        {
            US_A_Data = Get_US_A_CNT;//记录第二次捕获到的值
            if(US_A_Data>Last_US_A_Data) //计数器没有溢出的情况
            {
                ultrasonic.A = (US_A_Data - Last_US_A_Data) / 5750.f;
            }
            else //计数器已经溢出，需要加回溢出值
            {
                ultrasonic.A = (US_A_Data + 65535 - Last_US_A_Data) / 5750.0f;
            }
            if(ultrasonic.A>3.0f) ultrasonic.A = 3.0f;
            #if USE_KALMAN_FILTER
            ultrasonic.A = rangeA_KalmanFilter(ultrasonic.A);
            #endif
            #if USE_SLIDIING_FILTER
            ultrasonic.A = Mean_Filter_A(ultrasonic.A);
            #endif
            US_A_FLag=0; //状态位复原
            timeout_A=0; //超时时间复位
            Set_US_A_Rising;  //设置TIM5CH3为上升沿捕获
            END_A = 0;
        }
        else if(US_A_FLag==0)
        {
            Last_US_A_Data = Get_US_A_CNT; //记录第1次捕获到的值
            US_A_FLag = 2; //标记已经经过了第1次捕获
            Set_US_A_Falling; //设置TIM5CH3为下降沿捕获
        }
    }
    ////////////////  超声波A ////////////////
```

图 5-5 中断测距代码图示

```
void ReadUS_task(void *pvParameters)
{
    u32 lastWakeTime = getSysTickCnt();
    while(1)
    {
        // This task runs at a frequency of 200Hz (5ms control once)
        //此任务以100Hz的频率运行（5ms控制一次）
        vTaskDelayUntil(&lastWakeTime, F2T(RATE_200_HZ));

        rgb_100Hz_control++; //RGB频率控制走时

        //超声波超时时间监测
        //超时时间意义：超声波分组触发，当1组超声波未完成捕获时，另一组不会启动触发；故需要设置超时时间，防止单个超声波不工作时或被拔掉时影响其他超声波
        if(END_A==1) timeout_A++;
        if(END_B==1) timeout_B++;
        if(END_C==1) timeout_C++;
        if(END_D==1) timeout_D++;
        if(END_E==1) timeout_E++;
        if(END_F==1) timeout_F++;

        //S300机器人
        if(Car_Mode == S300)
        {
            //超声波ACE为1组一起触发
            if(!END_B&&!END_D&&!END_F&&!Trigger_Group_1)
            {
                Trigger_Group_1 = 1; //组1标记1次，在未完成组1的所有超声波任务前不会再进入此函数
                Trigger_Group_2 = 0; //完成了组2的任务，组2标记复位
                END_A = 1 , TriggerA = 1, delay_us(12) , TriggerA = 0 , timeout_A = 0;
                END_C = 1 , TriggerC = 1, delay_us(12) , TriggerC = 0 , timeout_C = 0;
                END_E = 1 , TriggerE = 1, delay_us(12) , TriggerE = 0 , timeout_E = 0;
            }

            //超声波BDF为1组一起触发
            if(!END_A&&!END_C&&!END_E&&!Trigger_Group_2)
            {
                Trigger_Group_2 = 1; //组2标记1次，未完成组2的所有超声波任务前不会再进入此函数
                Trigger_Group_1 = 0; //完成了组1的任务，组1标记复位
                END_B = 1 , TriggerB = 1, delay_us(12) , TriggerB = 0 , timeout_B = 0;
                END_D = 1 , TriggerD = 1, delay_us(12) , TriggerD = 0 , timeout_D = 0;
                END_F = 1 , TriggerF = 1, delay_us(12) , TriggerF = 0 , timeout_F = 0;
            }
```

图 5-6 超声波测距任务代码图示

5.3显示屏显示内容

OLED 显示屏在单击 User 按键时会切换不同的内容界面。下面为六个不同的显示内容界面。想要修改显示屏显示内容可以在 show.c 文件中的 oled_show()函数中修改。

下面介绍几个重要的显示函数，oled_showNumeber()函数是显示整数数据大小的，oled_showfloat()函数是显示小数数据大小的，则 oled_showString()函数是显示字符串的。则若想显示下面初始界面的第二行数据，则 OLED_ShowString(00,10,"GYRO_Z");OLED_ShowNumber(75,10,gyro[2],5,12);gyro[2]数据就是 Z 轴数据。

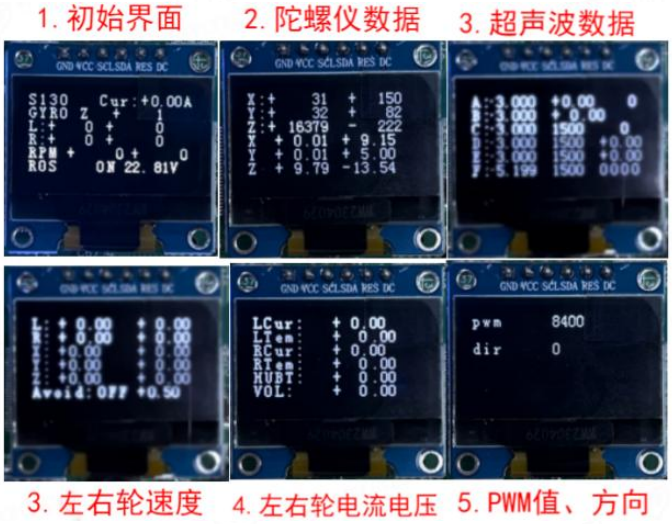


图 5-7 oled 显示屏显示内容图示

6. ROS 通信协议

本章内容主要说明 C63A 主板的 ROS 通信协议，如搭配我们公司的 ROS 机器人镜像，则不需要修改即可马上使用。通信协议分为发送数据与接收数据两个部分内容，详情请看下文。

6.1 ROS 接口控制输入协议说明

STM32 控制器板子上搭载了 CH9102(串口 4)串口通信接口，使用 CH9102(串口 4)和 ROS 端进行串口通信，以下为串口 4 接收数据说明。

接收数据采用中断接收的方式，接收的数据包括机器人产品信号、使能控制标志位、机器人三轴目标速度、数据校验位。

帧头帧尾默认是固定值；flag_stop 是机器人的使能控制位，默认发送使能；机器人三轴目标速度用于控制机器人运动，具体的接收内容见表 6-1。需要注意的是，表格中的数组号是上位机发送数据的数组号。

表 6-1 STM32 接收 ROS 发送过来的数据

数据内容	帧头(固定值 0X7B)	控制模式	安全策略标志位	机器人 X 轴目标速度		机器人 Y 轴目标速度		机器人 Z 轴目标速度		数据校验位	帧尾(固定值 0X7D)
数据类型	Uint8	Uint8	Uint8	short		short		short		Uint8	Uint8
占用字节	1	1	1	2		2		1		1	1
数组号	1	2	3	4	5	6	7	8	9	10	11

6.2ROS 接口默认发送的数据协议

ROS 和 STM32 控制器（运动底盘）之间通过串口实现通信，STM32 控制器使用的是串口 4，波特率是 115200。通信协议包括：STM32 控制器向 ROS 发送数据，ROS 向 STM32 控制器发送数据。打开 STM32 控制器的源码，关于串口通信的代码都在 STM32 控制器的 usartx.c 文件。

STM32 向 ROS 发送数据是按照 20Hz 的频率执行。发送的数据包括：帧头帧尾、机器人使能标志位、机器人 XYZ 三轴速度、IMU 三轴加速度、三轴角速度、电池电压、数据校验位,如果是 S300、S150、S130 等还需要发送超声波测距数据和自动回冲数据，详细发送的数据见下文的表 6-2。

发送数据的方式：将要发送的数据打包到一个数组中，数组的长度是 24 字节，使用串口逐位发送出去。因为串口一次只能发送一个 8 位（1 个字节）的数据，因此 2 个字节（short 型）的数据会拆分成高 8 位和低 8 位发送。

发送数据前对数据赋值的函数是“usartx.c”文件中的“data_transition( )”函数；发送数据的函数是“usartx.c”文件中的“UART4_SEND( )”函数。如果您需要更改发送的数据内容，直接更改 data_transition( )函数即可；如果您需要更改发送数据的长度，需要更改“buffer[ ]”这个数组长度的同时还需要“UART4_SEND( )”函数中 for 循环的次数，buffer[ ]这个数组长度修改 SEND_DATA_SIZE 即可，并且 ROS 端接收数据长度也要做相应的修改。

```
void data_transition(void)
{
    static u8 autorc_Timeout = 0; //用于检测CAN中断里是否还存在自动回充的数据
    static u8 RED;
    Send_Data.Sensor_Str.Frame_Header = FRAME_HEADER; //Frame_header //帧头
    Send_Data.Sensor_Str.Frame_Tail = FRAME_TAIL;     //Frame_tail //帧尾

    //Forward kinematics solution, from the current speed of each wheel to calculate the current speed of the three axis
    //运动学正解，从各车轮当前速度求出三轴当前速度
    Send_Data.Sensor_Str.X_speed = ((MOTOR_A.Encoder+MOTOR_B.Encoder)/2)*1000; //小车x轴速度
    Send_Data.Sensor_Str.Y_speed = 0;
    Send_Data.Sensor_Str.Z_speed = ((MOTOR_B.Encoder-MOTOR_A.Encoder)/Wheel_spacing)*1000;//小车z轴速度

#if OLED_DEBUG_MODE
    pos_x += ((Send_Data.Sensor_Str.X_speed/1000.0f)*cos(pos_z) - (Send_Data.Sensor_Str.Y_speed/1000.0f)*sin(pos_z))*0.05f;
    pos_y += ((Send_Data.Sensor_Str.X_speed/1000.0f)*sin(pos_z) + (Send_Data.Sensor_Str.Y_speed/1000.0f)*cos(pos_z))*0.05f;
    pos_z += (Send_Data.Sensor_Str.Z_speed/1000.0f)*0.05f;
    if(pos_x>100 || pos_x<-100) pos_x=0;
    if(pos_y>100 || pos_y<-100) pos_y=0;
    if(pos_z>100 || pos_z<-100) pos_z=0;//防止变量溢出，一般的debug也不需要用到这么大的数值
#endif

    //The acceleration of the triaxial acceleration //加速度计三轴加速度
    Send_Data.Sensor_Str.Accelerometer.X_data= accel[1]; //The accelerometer Y-axis is converted to the ros coordinate X axis //加速度计Y轴转换到ROS坐标X轴
    Send_Data.Sensor_Str.Accelerometer.Y_data=-accel[0]; //The accelerometer X-axis is converted to the ros coordinate y axis //加速度计X轴转换到ROS坐标Y轴
    Send_Data.Sensor_Str.Accelerometer.Z_data= accel[2]; //The accelerometer Z-axis is converted to the ros coordinate Z axis //加速度计Z轴转换到ROS坐标Z轴

    //The Angle velocity of the triaxial velocity //角速度计三轴角速度
    Send_Data.Sensor_Str.Gyroscope.X_data= gyro[1]; //The Y-axis is converted to the ros coordinate X axis //角速度计Y轴转换到ROS坐标X轴
    Send_Data.Sensor_Str.Gyroscope.Y_data=-gyro[0]; //The X-axis is converted to the ros coordinate y axis //角速度计X轴转换到ROS坐标Y轴
```

图 6-1 data_transition()函数

```
#ifndef __USRATX_H
#define __USRATX_H

#include "stdio.h"
#include "sys.h"
#include "system.h"

#define DATA_STK_SIZE   512
#define DATA_TASK_PRIO  4

#define FRAME_HEADER      0X7B //Frame_header //帧头
#define FRAME_TAIL        0X7D //Frame_tail   //帧尾
#define SEND_DATA_SIZE    24
#define RECEIVE_DATA_SIZE 11
```

图 6-2 SEND_DATA_SIZE 位置

在发送的数据中需要说明的是，其中帧头是是固定值 0X7B，帧尾是固定值 0X7D，flag_stop 是电机的停止标志位（0 是使能、1 是失能）。数据校验位的计算方式是 BCC 校验（将全部数据位（包括帧头）异或），最终得到的结果就是

数据校验位，数据校验位的计算过程见图 6-3。

```
u8 Check_Sum(unsigned char Count_Number,unsigned char Mode)
{
    unsigned char check_sum=0,k;

    //Validate the data to be sent
    //对要发送的数据进行校验
    if(Mode==1)
        for(k=0; k<Count_Number; k++)
        {
            check_sum=check_sum^Send_Data.buffer[k];
        }

    //Verify the data received
    //对接收到的数据进行校验
    if(Mode==0)
        for(k=0; k<Count_Number; k++)
        {
            check_sum=check_sum^Receive_Data.buffer[k];
        }

    //对要发送的超声波数据进行校验
    if(Mode==3)
    {
        for(k=0; k<Count_Number; k++)
        {
            check_sum=check_sum^Distance_Data.buffer[k];
        }
    }
    return check_sum;
}
```

图 6-3 数据校验位计算函数

表 6-2 STM32 向 ROS 发送的数据

数据内容	帧头(固定值0X7B)	flag_stop	机器人X轴速度		机器人Y轴速度		机器人Z轴速度		加速度计X轴加速度		加速度计y轴加速度	
数据类型	Uint8	Uint8	short		short		short		short		short	
占用字节	1	1	2		2		2		2		2	
数组号	1	2	3	4	5	6	7	8	9	10	11	12
数据内容	加速度计z轴加速度	角速度计X轴角速度	角速度计Y轴角速度		角速度计Z轴角速度		电池电压		数据校验位		帧尾(固定值0X7D)	
数据类型	short	short	short		short		short		Uint8		Uint8	
占用字节	2	2	2		2		2		1		1	

数组号	13	14	15	16	17	18	19	20	21	22	23	24

表 6-3 STM32 向 ROS 发送超声波等数据

数据内容	帧头(固定值 0XFA)	超声波 A		超声波 B		超声波 C		超声波 D		超声波 E		超声波 F	
数据类型	Uint8	short		short		short		short		short		short	
占用字节	1	1		2		2		2		2		2	
数组号	1	2	3	4	5	6	7	8	9	10	11	12	13
数据内容	机器人自检参数	机器人自检参数		机器人自检参数		机器人自检参数		数据校验位		帧尾(固定值 0XFC)			
数据类型	short	short		short		short		short		Uint8			
占用字节	1	1		1		1		1		1			
数组号	14	15		16		17		18		19			

这里还有一个地方需要注意，机器人 XYZ 三轴速度、加速度计、角速度计、电池电压以及超声波的原始数据是浮点型的数据（float），因为浮点型数据使用串口传输不方便，所以这四个数据在发送之前先将浮点数放大一千倍（保留小数点后三位），再将放大后的浮点数强制转换成 short 型数据，最后在发送前将 short 型数据拆分成两个 8 位的数据。相应的，上位机端在接收到数据后，需要将接收到的数据两个 8 位数据合并后转换为 short 型，在缩小一千倍来进行单位的转换。

下面讲解一下如何将两个 8 位数据合并后转换为 short 型，即得到我们的实际速度等数据：我们的控制量单位是 mm/s(0.001m/s)，控制量方向由高 8 位数据

的最高位决定。

示例 1：21 B6=0010 0001 1011 011 ，最高位为 0，正数，速度大小为 21 B6=(2*16+1)*256+(B*16+6)=(2*16+1)*256+(11*16+6)=8630(mm/s)；

示例 2：A1 2F=1010 0001 0010 1111，最高位为 1，负数，速度大小为 2^16(FF FF+1)-A1 2F=5E D0+1=(5*16+E)*256+(D*16+0)+1=24272(mm/s)。

下图为我们连接串口 4 后通过串口助手接收到的实际数据。

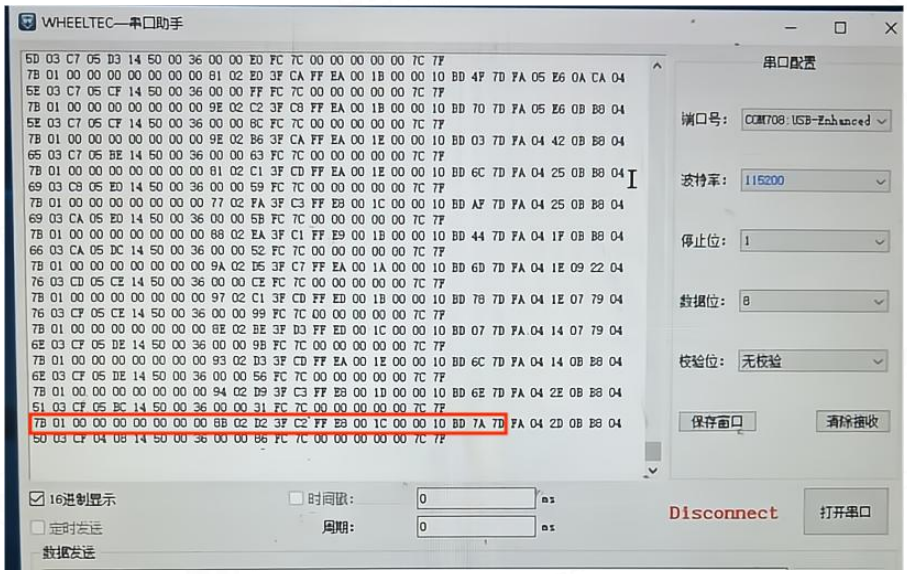


图 6-4 C63A 串口 4 发送的数据

下面对这接收到的 24 个字节数据转换一下：

第 1 个字节：0X7B，帧头；

第 2 个字节：0X01，电机处于停止状态；

第 3、4 个字节：X 轴速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X00(16 进制)=0000 0000(2 进制)，最高位为 0，正数(前进)，速度大小为 0*256+0=0(mm/s)，该速度为小车对编码器数据进行计算得到的实际速度。

第 5、6 个字节：Y 轴速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X00(16 进制)=0000 0000(2 进制)，最高位为 0，正数(左移)，速度大小为 0*256+0=0(mm/s)，该速度为小车对编码器数据进行计算得到的实际速度。

第 7、8 个字节：Z 轴速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X00(16 进制)=0000 0000(2 进制)，最高位为 0，正数(逆时针旋转)，速度大小为 0*256+0=0(0.001rad/s)，该速度为小车对编码器数据进行计算得到的实际速度。

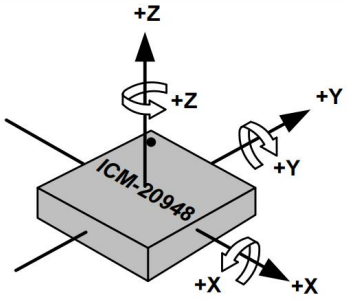


图 6-5 ICM2094 加速度计、角速度计三轴示意图

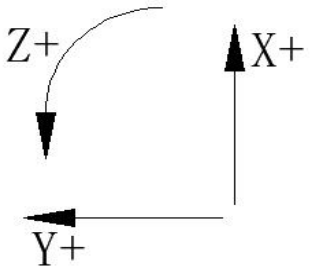


图 6-6 机器人 XYZ 三轴方向标定

接下来 12 个字节为三轴加速度计和角速度计的数据，注意这里 XYZ 三轴方向有所改变，X 轴正方向为向右，Y 轴正方向为向前，Z 轴正方向为上升。加速度计和角速度计的速度皆为绕 XYZ 三轴旋转的速度，如上图 6-2-4 所示。因为陀螺仪的 XY 轴方向与我们标定的机器人 XY 轴方向是有差别的，因此在 XY 轴的角速度和角速度数据在发送做了一个矫正的转换。

同时在 STM32 程序里面设置了 ICM2094 的加速度计量程为±2G=±2*9.8m/s^2=±19.6m/s^2，角速度计的量程为±500°/s=±8.72665rad/s。即例如读取计算得到 X 轴的加速度计数据为+32768，则代表此时机器人的前进加速度为+2G=+19.6m/s^2，32768/19.6=1672，则加速度数据转换为 m/s^2 的单位的比例参数为 1672。同理角速度数据转换为 rad/s 的单位的比例参数为 32768/8.72665=3755。

第 9、10 个字节：X 轴加速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X8B(16 进制)=1000 1011(2 进制)，最高位为 0，正数，大小为 008B=0*256+139=139，转换为加速度 139/1672=0.0831(m/s^2)。

第 11、12 个字节：Y 轴加速度，高 8 位 0X02(16 进制)=0000 0010(2 进制)、低 8 位 0XD2(16 进制)=1101 0010(2 进制)，最高位为 0，正数，大小为 02D2=2*256+210=722，转换为加速度 722/1672=0.4318(m/s^2)。

第 13、14 个字节：Z 轴加速度，高 8 位 0X3F(16 进制)=0011 FFFF(2 进制)、低 8 位 0XC2(16 进制)=1100 0010(2 进制)，最高位为 0，正数，大小为 63*256+194=16322，转换为加速度 16322/1672=9.7619(m/s^2)。

第 15、16 个字节：X 轴角速度，高 8 位 0XFF(16 进制)=1111 1111(2 进制)、低 8 位 0XE8(16 进制)=11101000(2 进制)，最高位为 1，负数，大小为 2^16(FF

FF+1)-FF E8=00 04=0*256+24=24，转换为角速度 24/3753=0.0063(rad/s)。

第 17、18 个字节：Y 轴角速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X1C(16 进制)=0001 1100(2 进制)，最高位为 0，正数，大小为 0*256+28=28，转换为角速度 28/3753=0.0074(rad/s)。

第 19、20 个字节：Z 轴角速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X00(16 进制)=0000 0000(2 进制)，最高位为 0，正数，大小为 0*256+0=0，转换为角速度 0/3753=0(rad/s)。

第 21、22 个字节：电池电压，高 8 位 0X10(16 进制)=0000 0001(2 进制)、低 8 位 0XBD(16 进制)=1011 1101(2 进制)，最高位为 0，正数，大小为 01*256+189=445，电压大小为 445mv(毫伏)。

第 23 字节：BCC 校验位(前 22 字节异或)，0X7A=0X7B^0X01^0X00^0X00^0X00^0X00^0X00^0X00^0X00^0X8B^0X02^0XD2^0X3F^0XC2^0XFF^0XE8^0X00^0X1C^0X00^0X00^0X10^0XBD。

（如果您想验证这个检验位的结果，可以使用一些网页在线平台来计算校验位）

第 24 个字节：0X7D，帧尾；

下面超声波数据的结算：

下图为我们连接串口 4 后通过串口助手接收到的实际数据。

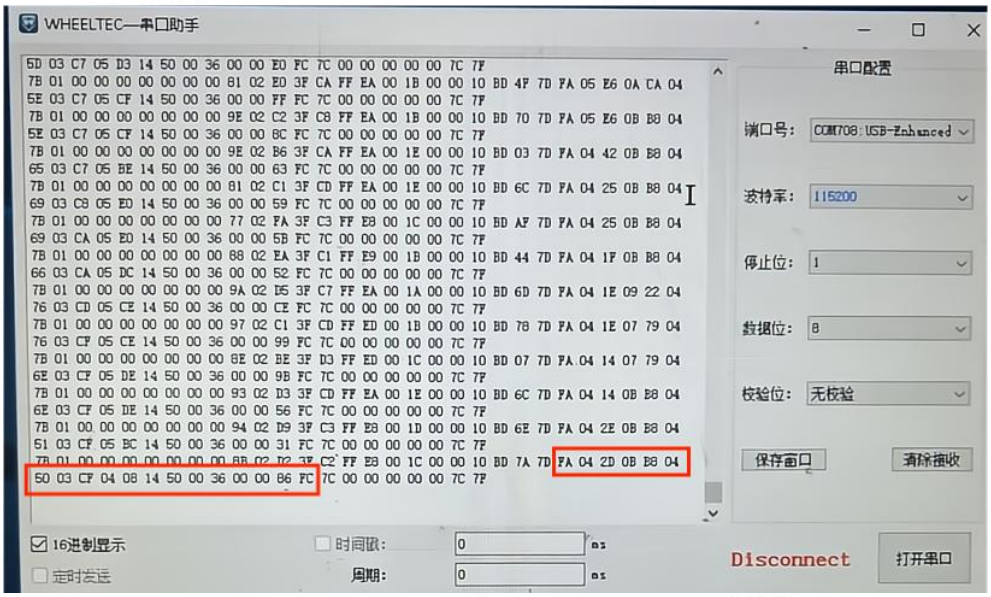


图 6-7 C63A 串口 4 发送的数据

下面对这接收到的 19 个字节数据转换一下：

第 1 个字节：0XFA，帧头；

第 2、3 个字节：超声波 A 数据，高 8 位 0X04(16 进制)=0000 0100(2 进制)、低 8 位 0X2D(16 进制)=0010 1101(2 进制)，距离大小为 4*256+45=1069(mm)，该距离为超声波 A 对最近物体进行测量得到的实际距离。

第 4、5 个字节：超声波 B 数据，高 8 位 0X0B(16 进制)=0000 1011(2 进制)、低 8 位 0XB8(16 进制)=1011 1000(2 进制)，距离大小为 11*256+184=3000(mm)，该距离为超声波 B 对最近物体进行测量得到的实际距离。

第 6、7 个字节：超声波 C 数据，高 8 位 0X04(16 进制)=0000 0100(2 进制)、低 8 位 0X50(16 进制)=0101 0000(2 进制)，距离大小为 4*256+80=1104(mm)，该距离为超声波 C 对最近物体进行测量得到的实际距离。

第 8、9 个字节：超声波 D 数据，高 8 位 0X03(16 进制)=0000 0011(2 进制)、低 8 位 0XCF(16 进制)=1100 1111(2 进制)，距离大小为 3*256+207=975(mm)，该距离为超声波 D 对最近物体进行测量得到的实际距离。

第 10、11 个字节：超声波 E 数据，高 8 位 0X14(16 进制)=1010 0100(2 进制)、低 8 位 0X50(16 进制)=0101 0000(2 进制)，距离大小为 14*256+80=3664(mm)，该距离为超声波 E 对最近物体进行测量得到的实际距离。

第 12、13 个字节：超声波 F 数据，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X36(16 进制)=0011 0110(2 进制)，距离大小为 0*256+54=54(mm)，该距离为超声波 F 对最近物体进行测量得到的实际距离。

第 14、15、16、17 个字节：机器人自检参数，可忽略。

第 18 个字节：BCC 校验位(前 22 字节异或)，0X86=0XFA^0X04^0X2D^0X0B^0XB8^0X04^0X50^0X03^0XCD^0X04^0X08^0X14^0X50^0X00^0X36^0X00^0X00。

（如果您想验证这个检验位的结果，可以使用一些网页在线平台来计算校验位）

第 19 个字节：0XFC，帧尾。

7. 如何给 STM32 控制器下载代码

程序下载可以使用串口一键下载，或使用 ST-Link 进行下载。

7.1 串口一键下载

串口一键下载步骤如下：

①硬件准备：主板与 TYPT-C 手机数据线

②软件准备：软件使用 MCUISP 烧录软件，在此之前先安装 USB 转 TTL 模块 CH9102 的驱动，附送的资料里面有驱动安装程序。驱动安装成功用数据线连接小车，打开电脑的设备管理器可以查看驱动是否安装成功。

图 7-1 驱动安装成功图示

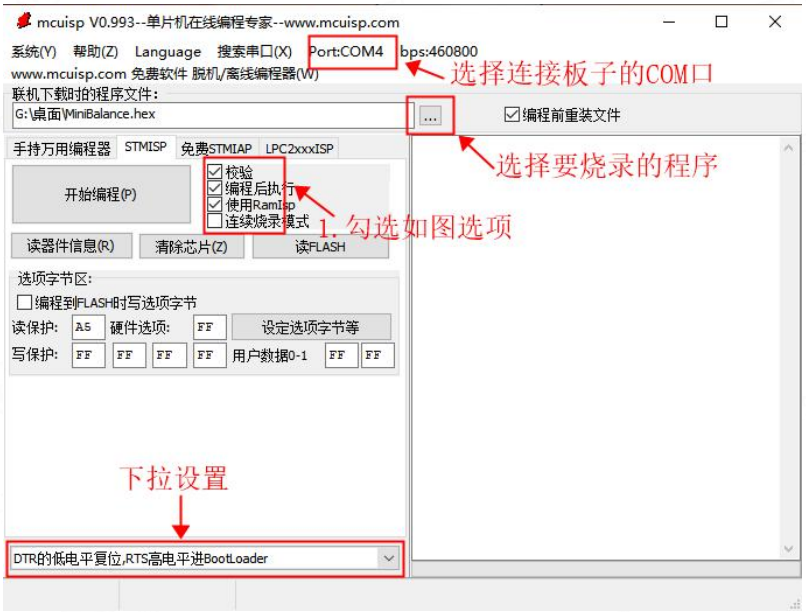


图 7-2 ISP 软件设置

从图 7-1 我们可以看到驱动已经安装成功，否则就会有红色感叹号。

③连接小车，打开附送资料的 MCUISP 软件，进行如图 7-2 的设置。程序文件在相应的工程文件下的 Output 文件夹里，后缀为.hex。

一切准备就绪，然后点击开始编程，程序就可以下载了。因为勾选了编程后执行，所以程序下载后会自动运行。

7.2 ST-Link 下载

使用 ST-Link 下载步骤如下：

1.硬件准备：ST-Link 下载器

2. 软件准备：Keil 编译器和 ST-Link 驱动程序。程序安装完成后连接 ST-Link 烧录器，打开电脑的设备管理器，点击通用串行总线设备会出现图 7-3 所示内容，表明安装成功。

图 7-3 驱动成功安装图示

3.安装完成后，安装线序连接板子和 ST-Link，板子上的 ST-Link 连接接口在右上角排针的最后一排。

图 7-4 ST-Link 下载接口

Keil 软件需要进行设置，才能使用 ST-Link 烧录器。点击魔术棒-Debug 选项卡，进行图 7-5 和 7-6 所示设置。

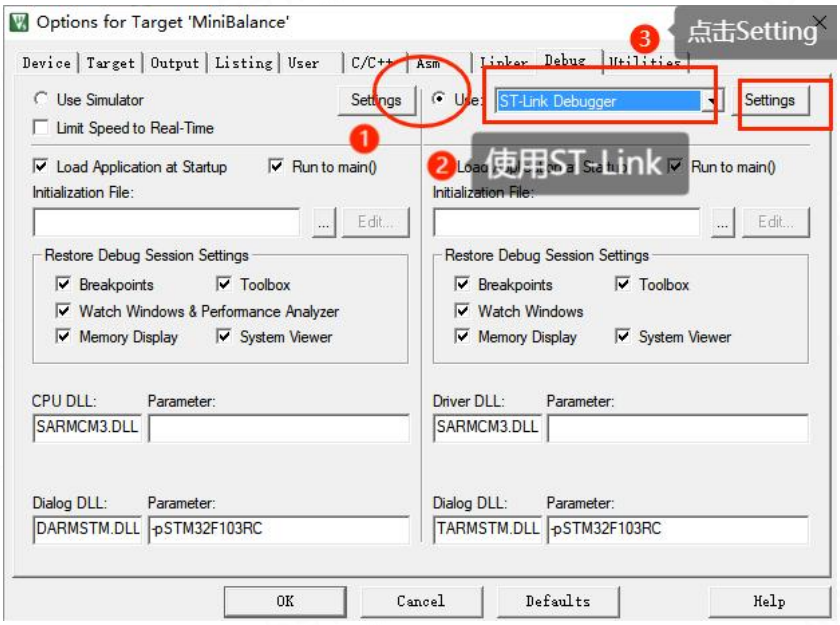


图 7-5 ST-Link 设置 1

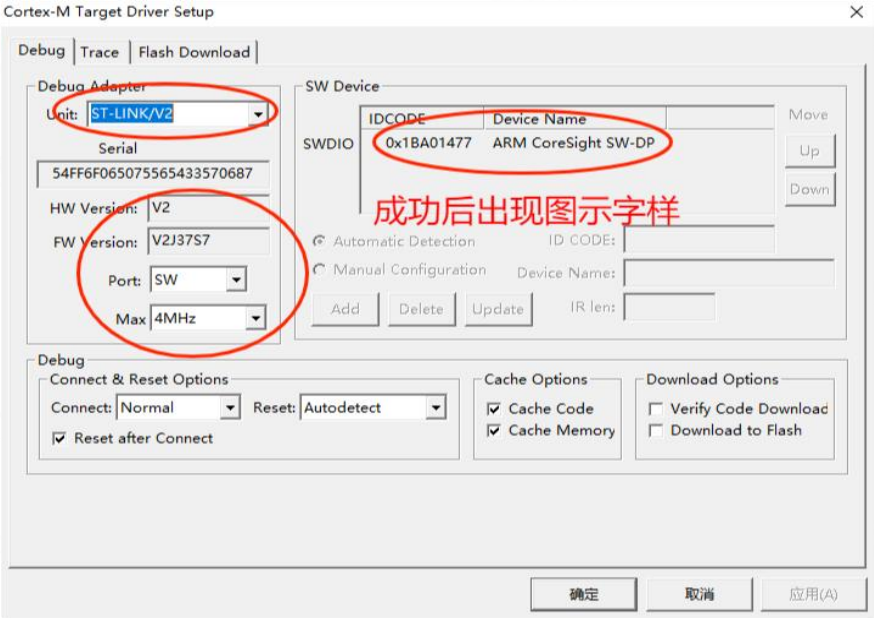


图 7-6 ST-Link 设置 2

设置完成后，重新编译，然后点击图 7-7 所示的下载按键即可下载到小车上

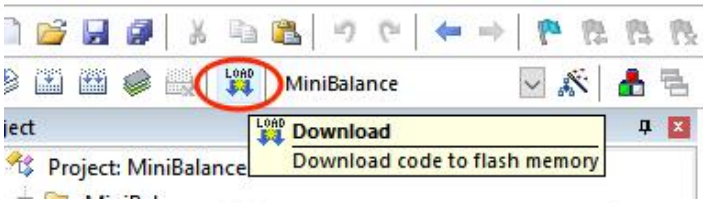


图 7-7 下载按键